

Informe de Calidad

Simulador Mundial 2026

Revision, Triage y Correccion (RTC)

Generado: 2026-06-18 18:58

Estado: APROBADO - Todos los Quality Gates

1. Resumen Ejecutivo

Este informe documenta el ciclo completo de aseguramiento de calidad para el proyecto Simulador Mundial 2026. Partiendo de un estado inicial con 4 tests funcionales fallando (severidad BLOQUEANTE) y 10 errores de linting con Ruff, el proyecto atravesó fases sistémicas de Revisión, Triage y Corrección (RTC) hasta alcanzar el cumplimiento total.

El proceso de corrección incluyó validaciones de esquema en backend, bugs de JavaScript en frontend, endurecimiento de seguridad CSS y aplicación del Quality Gate de SonarQube. Cada defecto identificado fue trazado, corregido y verificado.

Métricas Clave

Tests totales:	83 (100% OK)
Bugs encontrados y corregidos:	6
Code smells corregidos:	7
Errores Ruff:	10 -> 0
Cobertura SonarQube:	95.7% en código nuevo
Quality Gate:	APROBADO
Estado Final:	GO para Release

2. Reporte de Bugs - Validaciones Backend

Se identificaron cuatro defectos funcionales de severidad BLOQUEANTE en el endpoint POST /teams/. El backend aceptaba datos invalidos (nombres vacios, codigos de longitud incorrecta) y persistia entidades inconsistentes. Los cuatro fueron causados por la ausencia de validaciones en el esquema Pydantic.

BUG-01 BLOQUEANTE

Descripcion:Codigo demasiado corto (2 caracteres)

Acepto codigo 'AR' (largo=2) devolviendo 201 en lugar de 422

Correccion: Se agrego Field(min_length=3, max_length=3) + field_validator en schemas/team.py

BUG-02 BLOQUEANTE

Descripcion:Codigo demasiado largo (4 caracteres)

Acepto codigo 'ARGN' (largo=4) devolviendo 201 en lugar de 422

Correccion: El mismo Field(min_length=3, max_length=3) cubre este caso

BUG-03 BLOQUEANTE

Descripcion:Codigo vacio

Acepto codigo " devolviendo 201 en lugar de 422

Correccion: field_validator limpia espacios y rechaza strings vacios

BUG-04 BLOQUEANTE

Descripcion:Nombre vacio

Acepto nombre " devolviendo 201 en lugar de 422

Correccion: field_validator('name') con strip + verificacion de no vacio

3. Reporte de Bugs - Frontend JavaScript

Se descubrieron dos bugs adicionales en el código de visualización del bracket durante el análisis de SonarQube. Los operadores condicionales devolvían el mismo valor en ambas ramas, deshabilitando efectivamente el resaltado visual del ganador en los partidos de Octavos y Cuartos de Final.

BUG-05**IMPORTANTE****Clase winner del bracket siempre vacía (Octavos + Cuartos)**

El ternario `${m.winner === m.home_team ? " : "}` devolvía string vacío en ambas ramas

Corrección: Cambiado a `${m.winner === m.home_team ? 'winner' : "}`

BUG-06**IMPORTANTE****Mismo bug en el equipo visitante del bracket**

El ternario `${m.winner === m.away_team ? " : "}` devolvía string vacío en ambas ramas

Corrección: Cambiado a `${m.winner === m.away_team ? 'winner' : "}`

Nota: Los widgets de Tercer Puesto y Final ya estaban correctos (usaban la clase 'winner'). Solamente los partidos de rondas genéricas (Octavos, Cuartos, Semis) tenían el bug.

4. Code Smells Corregidos

4.1 Linting con Ruff (10 errores)

El analisis de Ruff encontro 10 errores de linting en todo el codigo. Se abordaron en dos pasadas: 7 auto-corregidos (imports no usados F401, f-strings F541) y 3 corregidos manualmente.

- [F401] Imports no utilizados en 5 archivos -> Auto-corregido con ruff check --fix
- [F541] F-string sin placeholders en 2 archivos -> Auto-corregido
- [F841] Variables no utilizadas en 2 archivos -> Manual: services/simulator_service.py, tests/test_dashboard.py
- [E402] Import no al inicio del archivo en 1 archivo -> Manual: schemas/team.py (import PlayerResponse)

4.2 Code Smells de SonarQube (7 en static/index.html)

SonarQube identifico 7 code smells en el archivo frontend HTML/JS. Todos fueron corregidos:

- Seguridad SRI (3x) - Faltaban atributos integrity en CDN de Bootstrap -> Se agregaron hashes sha384 + crossorigin='anonymous'
- Ternario Anidado (2x) - Los badges de posicion usaban operadores ternarios anidados -> Extraidos a funcion auxiliar getPositionClass()
- Bloque Catch Vacio (1x) - El catch del dashboard silenciaba errores sin registrar -> Se agrego console.warn con mensaje de error
- Ternario Mismo Valor (1x) - El bracket tenia ramas true/false identicas -> Corregido para usar clase CSS 'winner' (cubierto en seccion Bugs)

4.3 Endurecimiento de Google Fonts

El enlace externo a la hoja de estilos de Google Fonts fue reemplazado con declaraciones @font-face embebidas. Esto elimino la vulnerabilidad restante de Integridad de Subrecursos (SRI) manteniendo la compatibilidad tipografica completa. Se agregaron 7 bloques @font-face para Inter (400, 500, 600, 700) y Poppins (500, 600, 700).

5. Resultados del Quality Gate

SonarQube fue configurado con el quality gate predeterminado 'Sonar way'. El proyecto fue analizado tres veces a medida que los problemas se resolvían progresivamente:

Analisis	Violac.	Cobert.	Duplic.	Result.
Inicial (pre-fix)	8	100.0%	0.0%	FALLIDO
Schema + ternarios	4	95.7%	0.0%	FALLIDO
Correccion completa	0	95.7%	0.0%	APROBADO

FINAL: APROBADO

Con todas las condiciones cumplidas (95.7% de cobertura, 0% de duplicación, 0 violaciones nuevas), el proyecto pasa el Quality Gate de SonarQube y queda autorizado para su lanzamiento.

6. Archivos Modificados

[schemas/team.py](#)

Se agrego `Field(min_length=3, max_length=3)` en `code`, `field_validators` para normalizacion del codigo (strip, mayusculas, exactamente 3 caracteres) y validacion del nombre (strip, no vacio). Se importaron `Field` y `field_validator`.

[static/index.html](#)

Se reemplazaron 2 ternarios anidados con la funcion auxiliar `getPositionClass()`. Se corrigieron 2 bugs de ternario en bracket (`? " : " -> ? 'winner' : ''`). Se agregaron hashes de integridad SRI a los enlaces CDN de Bootstrap CSS y JS. Se reemplazo el enlace externo de Google Fonts con bloques `@font-face` embebidos. Se agrego `console.warn` al bloque `catch` vacio.

[tests/test_teams.py](#)

Se agregaron 9 nuevos tests de validacion (TEAMS-01 a TEAMS-09) incluyendo cobertura para codigo corto/largo/vacio, nombre vacio, campos faltantes, tipos incorrectos, duplicados case-insensitive y normalizacion a mayusculas.

[tests/test_frontend_smoke.py](#)

Se creo suite de smoke test (SMK-01 a SMK-06) con 35 verificaciones automatizadas que cubren elementos UI, integracion con API, estados de carga, validacion de contenido y layout responsive.

[qa_report.json](#)

Exportacion de hallazgos de Ruff para analisis.

[coverage.xml](#)

Regenerado con 83 tests exitosos (95.7% de cobertura en codigo nuevo).

[allure-report/](#)

Reporte HTML generado con decoradores Allure (83 tests).

7. Conclusion

El proyecto Simulador Mundial 2026 ha completado un ciclo completo de Revision, Triage y Correccion (RTC). Todos los problemas identificados fueron trazados, corregidos y verificados mediante pruebas de regresion automatizadas.

Que estaba mal:

- 4 bugs BLOQUEANTES en backend: POST /teams/ aceptaba datos invalidos
- 2 bugs IMPORTANTES en frontend: resaltado de ganador en bracket siempre deshabilitado
- 10 errores de linting Ruff (imports sin usar, variables, f-strings)
- 7 code smells en index.html (SRI, ternarios, catch vacio)
- 0% de cobertura de tests automatizados en frontend

Como se resolvio:

- Validaciones de esquema en Pydantic: codigo exactamente 3 caracteres, nombre no vacio
- Suite de smoke tests para frontend (35 verificaciones automatizadas)
- Ruff: 7 auto-corregidos + 3 correcciones manuales -> 0 errores
- SonarQube: hashes SRI, funciones extraidas, fuentes embebidas
- Regresion completa: 83/83 tests exitosos, cobertura 95.7%

Veredicto final:

GO para Release

Cero defectos Bloqueantes. Cero defectos Criticos. Los 83 tests pasan correctamente. Linting Ruff limpio (0 errores). Reporte Allure generado. Quality Gate de SonarQube: APROBADO (95.7% cobertura, 0% duplicacion, 0 violaciones nuevas).